

IF 45 I

Advance Web Programming

Meet8 - Introduction to Express.js

Wirawan Istiono, S.Kom., M.Kom



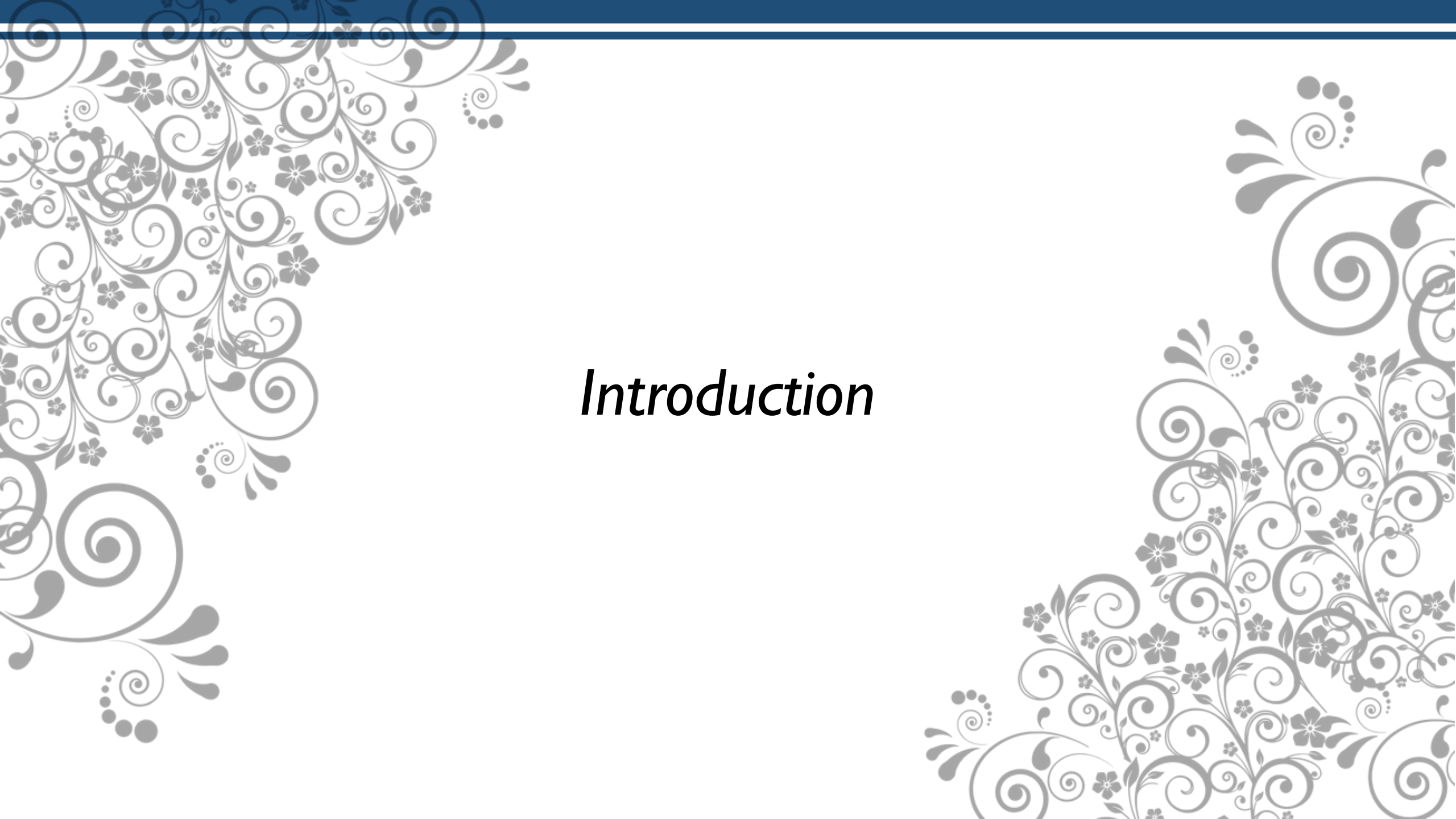
Meet 8

Objective:

Sub-CLO0714 - Students are able to implement Node JS Framework to build Web application (C3, C6)

OUTLINE

- *Installing Express.*
- *Route handling (GET, POST).*
- *Sending JSON responses*

The slide features a light gray background with decorative floral and scrollwork patterns in the corners. The top edge has a dark blue horizontal band. The word "Introduction" is centered in a black, italicized serif font.

Introduction

What is Express.js?

Express.js is a framework built on top of Node.js that makes building web applications easier. It simplifies routing, middleware, request/response handling, and integrates well with databases and templating engines.

Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.

So, when you use Express.js, you are still using Node.js — but with additional helper features.

Official website of Express.js : <https://expressjs.com/>

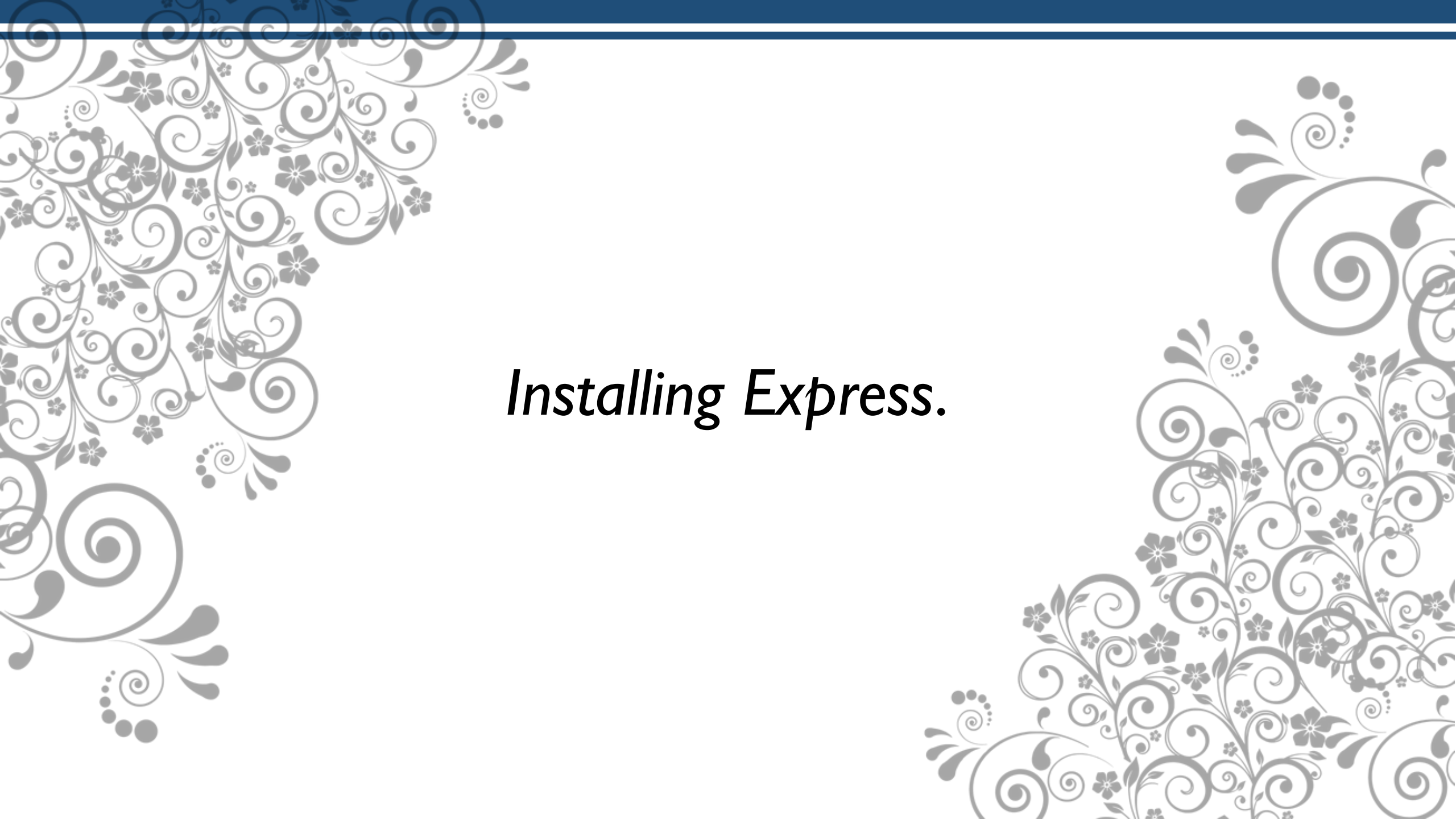


Comparison of Popular Node.js Frameworks (I)

Framework	Advantages	Disadvantages
Express.js	- Most popular and widely used (huge community & ecosystem) - Lightweight and minimal, gives freedom in structuring your app - Easy to learn, perfect for beginners - Tons of middleware and plugins available	- Minimal structure (you must organize everything yourself) - Can get messy in large-scale apps - No built-in support for TypeScript
Koa.js	- Created by the same team behind Express - Very lightweight, smaller core than Express - Uses modern JavaScript (async/await) out of the box - Better error handling than Express	- Smaller community than Express - Requires more setup (middleware not included by default) - Less beginner-friendly
NestJS	- Built with TypeScript by default (type safety) - Strong architecture (MVC, modular, dependency injection) - Great for large enterprise apps and microservices - Rich ecosystem (GraphQL, WebSockets, etc.)	- Steeper learning curve - Overhead for small/simple apps - Requires knowledge of TypeScript & decorators

Comparison of Popular Node.js Frameworks (2)

Framework	Advantages	Disadvantages
Fastify	- Extremely fast (better performance than Express) - Low overhead, optimized for speed - Built-in JSON schema validation - Plugin system for modular development	- Smaller ecosystem than Express - Not as widely adopted - Learning curve for schema-first approach
Hapi.js	- Strong focus on configuration over code - Powerful plugin system - Good for building secure, scalable applications - Rich built-in features (auth, validation, caching)	- More complex than Express - Less flexible (opinionated design) - Smaller community compared to Express
Sails.js	- Inspired by Ruby on Rails (MVC architecture) - Built-in ORM (Waterline) for database support - Very fast prototyping of APIs - Good for real-time apps (WebSockets support)	- ORM (Waterline) can be limiting - Heavier than other frameworks - Less popular now (community shrinking)



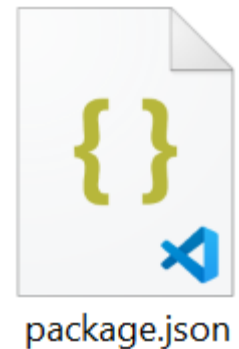
Installing Express.

Pre-installation Express

Before installing Express.js you need to following this step:

1. Create your folder and enter to your project folder
2. Open your terminal or command prompt
3. [Optional] Install npm init, to make your project is neater, dependencies are recorded in package.json, easy to share & maintain, with following command to create package.json.

npm init -y



Installing Express.js

To install Express.js, following this step:

1. Make sure your command in terminal is in your project folder
2. Type this command to install Express.js

npm install express

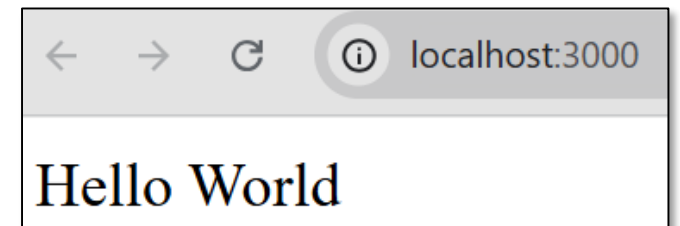
Basic Template Express

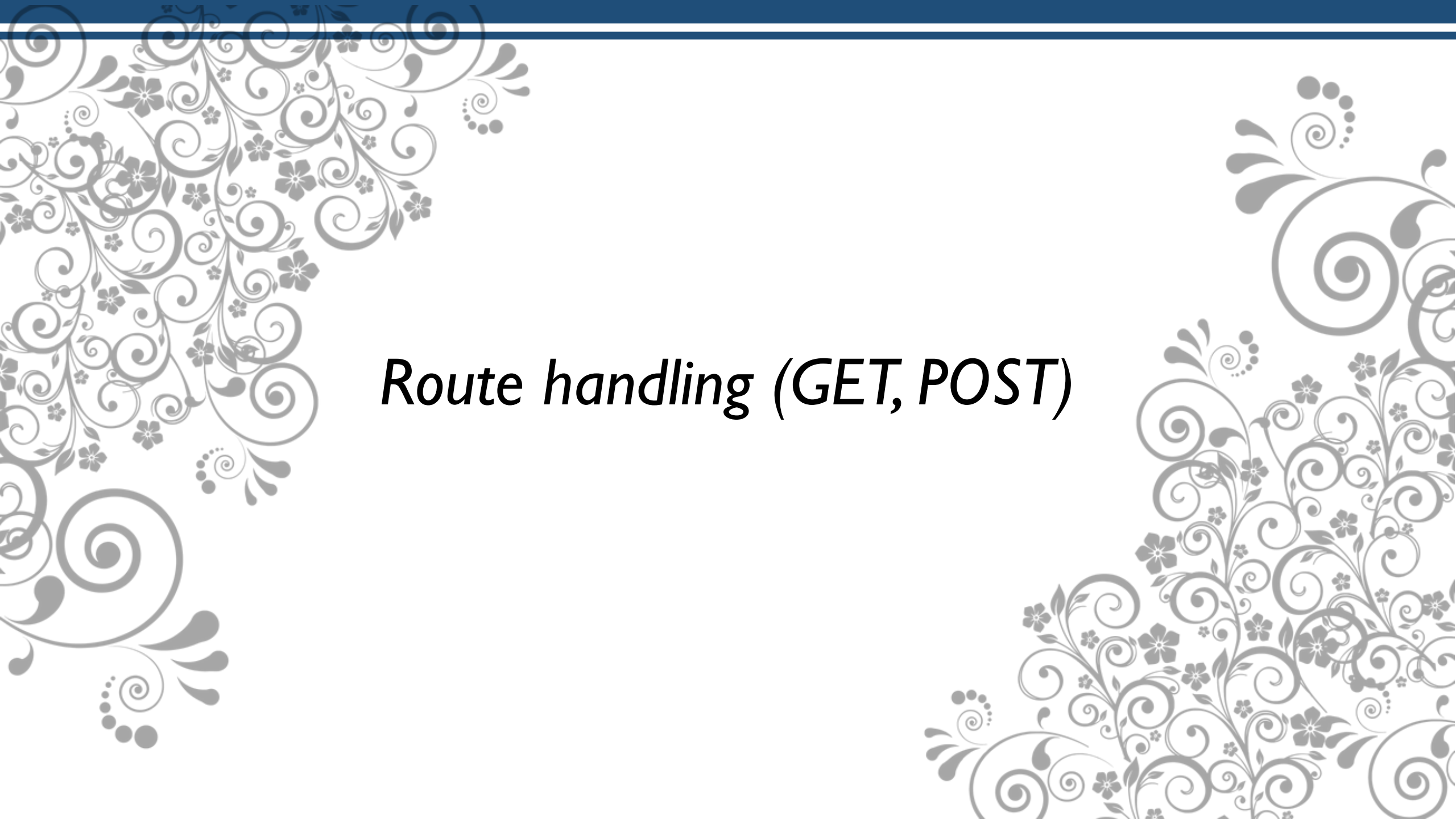
1. Create new file named **app.js** and put this code.
2. After that, run your app with command “**node app.js**” in your terminal
3. And See the result with your browser: <http://localhost:3000>

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(port, () => {
  console.log(`Server is running at http://localhost:${port}`);
});
```





Route handling (GET, POST)

What is Route Handling in ExpressJS?

In ExpressJS, route handling is the way you define how your server should respond to different HTTP requests (like GET and POST) for specific URLs (routes).

- GET route → used to fetch data or show a page. Example: when you visit `http://localhost:3000/` in your browser.
- POST route → used to send data to the server, often from a form submission or API request.

Think of routes as "rules" that say: "If a request comes to this URL using this method, then run this code."

Basic Route GET (I)

In this sample, we want to show like the sample below if we visit some url:

- “http://localhost:3000” then will show “Ini adalah halaman utama”
- “http://localhost:3000/home” then will show “Ini adalah halaman home”
- “http://localhost:3000/profile” then will show “Ini adalah halaman profile”

Basic Route GET (2)

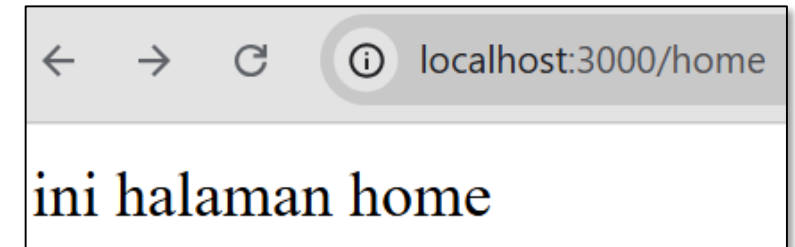
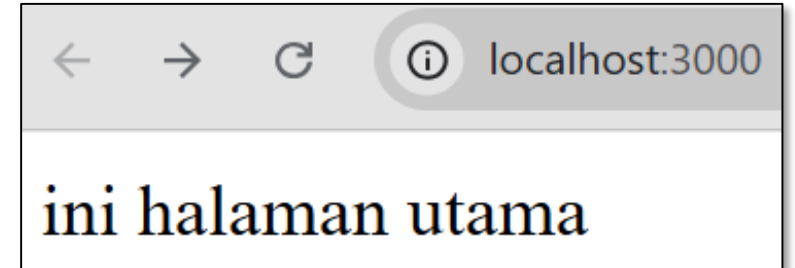
```
const express = require("express");
const app = express();
const port = 3000;

app.get("/", (req, res) => {
  res.send("ini halaman utama");
});

app.get("/home", (req, res) => {
  res.send("ini halaman home");
});

app.get("/profile", (req, res) => {
  res.send("ini halaman profile");
});

app.listen(3000, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```



Send parameter using GET

In ExpressJS, you can pass parameters in **two common ways**:

1. Query parameter:

`http://localhost:3000/home?nama=wirawan`

2. Route Parameters:

`http://localhost:3000/home/wirawan`

Send parameter using GET (I) – Query Parameter

If you have url like the sample below:

`http://localhost:3000/home?nama=wirawan&office=umn`

You can get parameter “nama” using this code:

`YourRequest.query.parameterName`



```
const express = require("express");
const app = express();

app.get("/home", (req, res) => {
  const nama = req.query.nama;
  const kampus = req.query.office;
  if(nama==undefined) {
    res.send("Halo Guest!");
  } else {
    res.send("Hello, " + nama + " dari " + kampus);
  }
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Send parameter using GET (2) – Route Parameter

If you have url like the sample below:

http://localhost:3000/home/wirawan

You can get parameter “wirawan” using this code, but in get URL, you need add “:yourParameterName”

YourRequest.query.parameterName

```
const express = require("express");
const app = express();

app.get("/home/:namaku", (req, res) => {
  const nama = req.params.namaku;
  res.send(`Welcome home, ${nama}!`);
});

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```



Send Parameter using POST – Prepare HTML file

First, you need to create new HTML page named “**meet8_form.html**”, and write code like the sample below:

```
<form method="POST" action="http://localhost:3000/submit">  
  Nim: <input type="text" name="txtNim" required><br>  
  Nama: <input type="text" name="txtNama" required><br>  
  Jurusan: <input type="text" name="txtJurusan" required><br>  
  <button type="submit">Kirim</button>  
</form>
```

Send Parameter using POST – Read HTML code

To call or read HTML code that had been made, you need:

1. Use “path” module
2. Using ***res.sendFile()***, this method is suitable when you want to serve a specific HTML file in response to a particular route

```
const express = require('express');
const app = express();
const path = require('path');

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'meet8_form.html'));
});

app.listen(3000, () => {
  console.log(`Server is running at http://localhost:3000`);
});
```

Note:

- **__dirname** refers to the directory where your server script is located.
- **path.join()** safely concatenates the directory path with the filename, ensuring correct path separators across different operating systems.
- **yourfile.html** should be a file in the same directory as your server script, or you can adjust the path accordingly.

Send Parameter using POST – Catch POST parameters

To access POST variables (data sent in the request body) in Express.js, you need to use:

1. Use **`express.urlencoded({ extended: true })`** in your express app.
2. “**`req.body.yourHTMLObjectName`**” object within your route handler

Edit or add your **`app.js`** script like the sample beside

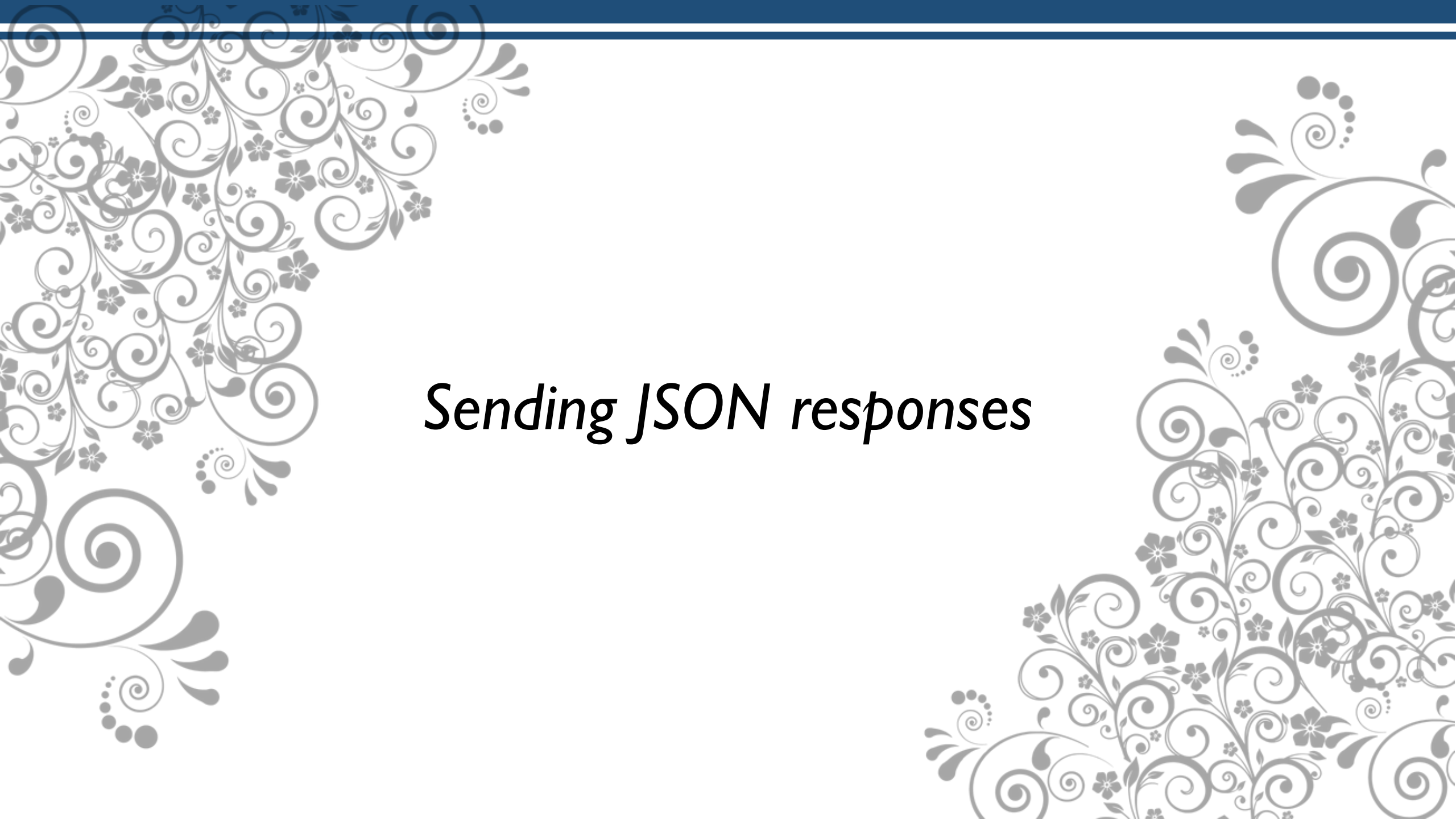
```
const express = require('express');
const app = express();
const path = require('path');

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'meet8_form.html'));
});
app.use(express.urlencoded({ extended: true }));

app.post('/submit', (req, res) => {
  const hasilRespon = "Nim:" + req.body.txtNim +
    "<br>Nama: " + req.body.txtNama +
    "<br>Jurusan: " + req.body.txtJurusan;

  res.send(hasilRespon);
});

app.listen(3000, () => {
  console.log(`Server is running at http://localhost:3000`);
});
```

The slide features a light gray background with decorative floral and scrollwork patterns in the corners. The top-left and bottom-right corners are filled with intricate gray designs, including swirls, leaves, and small flowers. A solid dark blue horizontal bar runs across the top of the slide.

Sending JSON responses

send JSON parameter

To change parameter to JSON, you just only need change and use **res.json**

```
const express = require('express');
const app = express();
const path = require('path');

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'meet8_form.html'));
});

app.use(express.urlencoded({ extended: true }));

app.post('/submit', (req, res) => {
  res.json(req.body);
});

app.listen(3000, () => {
  console.log(`Server is running at http://localhost:3000`);
});
```

NEXT WEEK'S OUTLINE

- Middleware & Routing in Express

REFERENCES

- B. Lawson and J. Encinas, Node.js Web Development, 6th ed., Packt Publishing, 2023
- R. Raj, Learning Node.js Development, Packt Publishing, 2018.
- E. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed., Manning Publications, 2017.
- Maulana, a., bau, r.t.r., munawar, z., setiawan, r., aisa, s., istiono, w., irfan, a., pratama, y.a., ramadhany, e.d., pomalingo, s. And permana, a.a., 2023. *Pemrograman web 101: memahami dasar-dasar untuk mengembangkan situs web*. Get press indonesia.
- The Node.js Foundation, “Node.js Documentation,” [Online].Available: <https://nodejs.org/en/docs/>. [Accessed: Jun. 5, 2025].
- Express.js Team, “Express.js Documentation,” [Online].Available: <https://expressjs.com/>. [Accessed: Jun. 5, 2025].

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.



Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.